

Student Performance Analyzer

Project Overview

The ****Student Performance Analyzer**** is a command-line Python application made to manage and analyze student performance.

This application lets users do the following:

- * Add student information
- * Enter marks for each subject
- * Record attendance
- * Generate a report for one student
- * Show all students who are registered
- * Look at the class performance
- * Calculate grades and performance levels
- * Find out which subject is the best and which is the weakest

The project shows how to use basic Python programming ideas like dictionaries, functions, loops, conditionals, error handling, input checks and data processing.

Student Details

****Name:**** Manas Pandey

****Registration ID:**** 26BCE10001

****Branch:**** CSE – Educational Technology

Technologies Used

* Python 3

* Command Line Interface (CLI)

* VS Code

No extra Python libraries are needed.

Features

1. Add Student

Users can register a student by giving:

- * Roll Number

- * Name

- * Branch

- * Semester

Each student is saved in a dictionary.

2. Add Marks

The app allows users to enter:

- * How subjects there are

- * Names of the subjects

- * Marks they got in each subject

Marks are checked only values from **0 to 100** are allowed.

3. Add Attendance

The application keeps track of:

- * Total number of classes held

- * Number of classes attended

Attendance percentage is calculated like this:

```
```text
```

Attendance Percentage =

$(\text{Classes Attended} / \text{Total Classes}) \times 100$

```
```
```

The program also makes sure that attendance data is valid.

4. Student Report

A full report can be created for any student who is registered.

The report includes:

- * Student name

- * Roll number

- * Branch

- * Semester

- * Marks for each subject

- * marks

- * Average percentage

- * Grade

- * Attendance percentage

- * Attendance status

- * Best subject

- * Weakest subject

- * Overall performance

5. Display Students

The app shows all students who are registered with:

- * Roll number

- * Name

- * Branch

6. Performance Analysis

The program looks at students who have marks entered and figures out:

* The student with the score

* The student with the score

* The average score for the class

Grading System

| Percentage | Grade |

| ----- | ---- |

90–100 | A+ |

| 80–89 | A |

70–79 | B+ |

| 60–69 | B |

| 50–59 | C |

| 40–49 | D |

| Below 40 | F |

Performance Classification

| Percentage | Performance | |
|------------|-------------|--|
|------------|-------------|--|

| ----- | ----- |
|-------|-------|
|-------|-------|

| | | |
|--------------|-----------|--|
| 75 and above | Excellent | |
|--------------|-----------|--|

| | | |
|-------|------|--|
| 60–74 | Good | |
|-------|------|--|

| | | |
|-------|---------|--|
| 50–59 | Average | |
|-------|---------|--|

| | | |
|-------|-------------------|--|
| 40–49 | Needs Improvement | |
|-------|-------------------|--|

| | | |
|----------|------|--|
| Below 40 | Poor | |
|----------|------|--|

Attendance Status

The program uses **75% attendance** as the limit.

* **75% or higher:** Good

* **Below 75%:** Warning

Project Structure

```
```text
```

Student-Performance-Analyzer/

|

|— student\_performance.py

|— README.md

```
```
```

```
---
```

Requirements

Before running the project make sure Python 3 is installed on your computer.

Check if Python is installed using:

```
```bash
```

```
python --version
```

```
```
```

```
```Bash
```

```
python3 --version
```

```
...
```

No additional packages or tools are required.

```

```

```
How to Run
```

```
Step 1: Clone the Repository
```

```
```bash
```

```
git clone https://github.com/<your-github-username>/<repository-name>.git
```

```
...
```

```
### Step 2: Open the Project Folder
```

```
```bash
```

```
cd Student-Performance-Analyzer
```

```
...
```

```
Step 3: Run the Program
```

```
```bash
```

```
python student_performance.py
```

```
```
```

If your system uses `python3` run:

```
```bash
```

```
python3 student_performance.py
```

```
```
```

```

```

```
How to Use
```

After starting the program you will see this menu:

```
```text
```

```
STUDENT PERFORMANCE ANALYZER
```

```
-----
```

```
1. Add Student
```

2. Add Marks

3. Add Attendance

4. Student Report

5. Display Students

6. Performance Analysis

7. Exit

...

Type the number for the action you want.

Example Workflow

```text

1. Add Student

2. Add Marks

3. Add Attendance

4. Student Report

...

First add the student. Then enter their marks and attendance. Finally generate the report to see the results.

---

## ## Python Concepts Demonstrated

This project uses programming ideas:

### ### Dictionaries

Student info and subject scores are kept in dictionaries.

### ### Functions

The code is split into functions like:

```
```python
```

```
add_student()
```

```
add_marks()
```

```
add_attendance()
```

```
student_report()
```

```
display_students()
```

performance_analysis()

...

Conditional Statements

`if` `elif` and `else` are used for:

- * Assigning a grade
- * Classifying performance
- * Checking user input
- * Setting attendance status

Loops

`while` and `for` loops are used to:

- * Keep showing the menu again
- * Enter multiple subjects
- * Work through student records

Exception Handling

`try-except` blocks handle cases where numbers are not entered correctly.

Lambda Functions

Lambda expressions help find the bottom performers during analysis.

Data Processing

Built-in functions such as:

```
```python
```

```
sum()
```

```
max()
```

```
min()
```

```
len()
```

```
```
```

are used to work with student data.

```
---
```

Limitations

now student data only stays in memory while the program runs.

When the program closes all data disappears because no file or database storage is used.

Also the app runs in the command line. Does not have a graphical interface.

Future Improvements

Possible updates include:

- * Saving student data to files
- * Using SQLite or another database
- * Adding a graphical user interface
- * Letting users search for a student
- * Allowing editing and deletion of student records
- * Creating PDF reports
- * Adding login and authentication
- * Drawing charts. Graphs
- * Giving more detailed class statistics

* Exporting reports, to CSV or Excel

##

The ****Student Performance Analyzer**** offers a command-line way to manage and analyze student academic information.

It uses Python skills to solve a real educational task and gives a base that can grow into a complete student management and analytics tool.

Author

****Manas Pandey****

****Registration ID:** 26BCE10001**

****Branch:** CSE – Educational Technology**